

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 18
“Mesin Abstrak”



DISUSUN OLEH:
ITRHOH ANGGUN PAMUNGKAS
109082500117
S1 IF-13-02

DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

Mesin abstrak merupakan model komputasi yang dibangun di atas model komputasi lain dengan menyediakan tipe data dan operasi-operasi dasar tertentu untuk menyelesaikan suatu masalah. Pada Modul 18, konsep mesin abstrak diterapkan pada kartu domino dan mesin karakter. Mesin domino menyediakan operasi seperti mengocok kartu, mengambil kartu, melihat gambar kartu, dan menghitung nilai kartu, sedangkan mesin karakter menyediakan operasi membaca karakter secara berurutan hingga mencapai penanda akhir data. Konsep ini bertujuan untuk menyederhanakan penyelesaian masalah dengan menyediakan operasi-operasi dasar yang dapat digunakan kembali dalam berbagai algoritma.

SOAL LATIHAN

Statement Skema Pemrosesan Sekuensial

1&2.

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

type Domino struct {
    Sisi1 int
    Sisi2 int
    Nilai int
    Balak bool
}

type Dominoes struct {
    Kartu  [28]Domino
    Jumlah int
}

// Membuat 1 set kartu domino
func buatDominoes() Dominoes {
    var d Dominoes
    idx := 0

    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
            d.Kartu[idx] = Domino{
                Sisi1: i,
                Sisi2: j,
                Nilai: i + j,
                Balak: i == j,
            }
            idx++
        }
    }

    d.Jumlah = 28
    return d
}

// Soal 1b
func kocokKartu(d *Dominoes) {
    for i := d.Jumlah - 1; i > 0; i-- {
        j := rand.Intn(i + 1)
        d.Kartu[i], d.Kartu[j] = d.Kartu[j], d.Kartu[i]
    }
}
```

```

// Soal 1c
func ambilKartu(d *Dominoes) Domino {
    if d.Jumlah == 0 {
        return Domino{}
    }

    d.Jumlah--
    return d.Kartu[d.Jumlah]
}

// Soal 1d
func gambarKartu(k Domino, suit int) int {
    if suit == 1 {
        return k.Sisi1
    }
    return k.Sisi2
}

// Soal 1e
func nilaiKartu(k Domino) int {
    return k.Nilai
}

// Soal 2a
func galiKartu(d *Dominoes, target Domino) {
    fmt.Println("\nMencari kartu yang memiliki suit sama
dengan:")
    fmt.Printf("(%d,%d)\n", target.Sisi1, target.Sisi2)

    for d.Jumlah > 0 {
        k := ambilKartu(d)

        if k.Sisi1 == target.Sisi1 ||
            k.Sisi1 == target.Sisi2 ||
            k.Sisi2 == target.Sisi1 ||
            k.Sisi2 == target.Sisi2 {

            fmt.Printf("Ditemukan kartu (%d,%d)\n", k.Sisi1,
k.Sisi2)
            return
        }
    }

    fmt.Println("Tidak ditemukan kartu yang cocok")
}

// Soal 2b
func sepasangKartu(k1, k2 Domino) bool {
    return nilaiKartu(k1)+nilaiKartu(k2) == 12
}

func main() {
    rand.Seed(time.Now().UnixNano())

    dominoes := buatDominoes()

```

```

fmt.Println("Jumlah kartu awal:", dominoes.Jumlah)

kocokKartu(&dominoes)

k1 := ambilKartu(&dominoes)
k2 := ambilKartu(&dominoes)

fmt.Printf("Kartu 1 : (%d,%d)\n", k1.Sisi1, k1.Sisi2)
fmt.Printf("Kartu 2 : (%d,%d)\n", k2.Sisi1, k2.Sisi2)

fmt.Println("Nilai kartu 1 :", nilaiKartu(k1))
fmt.Println("Nilai kartu 2 :", nilaiKartu(k2))

if k1.Balak {
    fmt.Println("Kartu 1 adalah balak")
}

if sepasangKartu(k1, k2) {
    fmt.Println("Total nilai kedua kartu = 12")
} else {
    fmt.Println("Total nilai kedua kartu ≠ 12")
}

galiKartu(&dominoes, k1)
}

```

Output:

The screenshot shows a Go IDE with the following components:

- Explorer:** Shows a project structure with files like `laprakmodul1&2`, `laprakmodul3`, `laprakmodul4`, `laprakmodul5`, `laprakmodul10`, `laprakmodul12`, `laprakmodul14`, `laprakmodul17`, and `laprakmodul...`.
- Source File:** Displays the Go code for `laprakmodul18`, which includes functions for creating, shuffling, and drawing dominoes.
- Terminal:** Shows the command `run "D:\alpro\ALPRO2\laprakmodul18\laprakmodul18.go"` and the output:


```

Jumlah kartu awal: 28
Kartu 1 : (0,6)
Kartu 2 : (1,4)
Nilai kartu 1 : 6
Nilai kartu 2 : 5
Total nilai kedua kartu = 12

Mencari kartu yang memiliki suit sama dengan:
(0,6)
Ditemukan kartu (5,6)
PS D:\alpro\ALPRO2>

```
- Output Window:** Displays the name `NAMA: ITRHOH ANGUN PAMUNGKAS` and the NIM `NIM: 109082500117`.

1.

1a. Tipe Data Domino dan Dominoes

```
type Domino struct {
    sisi1 int
    sisi2 int
    nilai int
    balak bool
}

type Dominoes struct {
    kartu [28]Domino
    jumlah int
}
```

Deskripsi Program:

Program membuat tipe data Domino untuk menyimpan kedua sisi kartu, nilai kartu, dan status balak. Tipe data Dominoes digunakan untuk menyimpan satu set kartu domino yang terdiri dari 28 kartu beserta jumlah kartu yang masih tersedia.

1b. Prosedur kocokKartu()

```
func kocokKartu(d *Dominoes) {
    for i := d.jumlah - 1; i > 0; i-- {
        j := rand.Intn(i + 1)
        d.kartu[i], d.kartu[j] = d.kartu[j], d.kartu[i]
    }
}
```

Deskripsi Program:

Prosedur ini digunakan untuk mengacak urutan kartu domino sehingga susunan kartu menjadi tidak dapat diprediksi.

1c. Fungsi ambilKartu()

```
func ambilKartu(d *Dominoes) Domino {
    d.jumlah--
    return d.kartu[d.jumlah]
}
```

Deskripsi Program:

Fungsi ini mengambil satu kartu dari tumpukan kartu dan mengurangi jumlah kartu yang tersisa.

1d. Fungsi gambarKartu()

```
func gambarKartu(k Domino, suit int) int {
    if suit == 1 {
        return k.sisi1
    }
    return k.sisi2
}
```

Deskripsi Program:

Fungsi ini digunakan untuk melihat gambar atau nilai pada sisi tertentu dari kartu domino.

1e. Fungsi nilaiKartu()

```
func nilaiKartu(k Domino) int {
    return k.sisi1 + k.sisi2
}
```

```
}
```

Deskripsi Program:

Fungsi ini digunakan untuk menghitung nilai kartu domino dengan menjumlahkan kedua sisi kartu.

2. .

2a. Prosedur galiKartu()

```
func galiKartu(d *Dominoes, target Domino) {  
    for d.jumlah > 0 {  
        k := ambilKartu(d)  
  
        if k.sisi1 == target.sisi1 ||  
           k.sisi1 == target.sisi2 ||  
           k.sisi2 == target.sisi1 ||  
           k.sisi2 == target.sisi2 {  
  
            fmt.Println("Kartu ditemukan:", k)  
            return  
        }  
    }  
}
```

Deskripsi Program:

Prosedur ini mengambil kartu satu per satu dari tumpukan sampai ditemukan kartu yang memiliki gambar atau suit yang sama dengan kartu target.

2b. Fungsi sepasangKartu()

```
func sepasangKartu(k1, k2 Domino) bool {  
    return nilaiKartu(k1)+nilaiKartu(k2) == 12  
}
```

Deskripsi Program:

Fungsi ini digunakan untuk memeriksa apakah jumlah nilai dua kartu domino sama dengan 12. Jika sama, fungsi mengembalikan nilai true, jika tidak maka false.

Statement Skema Pemrosesan Sekuensial

3.

Source Code:

```
package main
import "fmt"

type Domino struct {
    Sisi1 int
    Sisi2 int
}

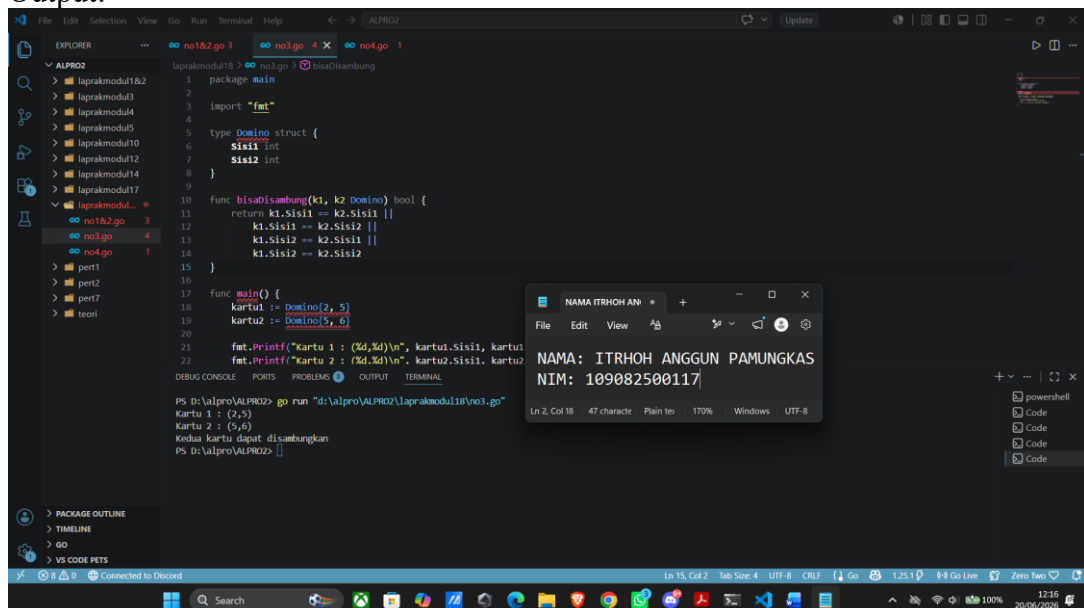
func bisaDisambung(k1, k2 Domino) bool {
    return k1.Sisi1 == k2.Sisi1 ||
        k1.Sisi1 == k2.Sisi2 ||
        k1.Sisi2 == k2.Sisi1 ||
        k1.Sisi2 == k2.Sisi2
}

func main() {
    kartu1 := Domino{2, 5}
    kartu2 := Domino{5, 6}

    fmt.Printf("Kartu 1 : (%d,%d)\n", kartu1.Sisi1,
kartu1.Sisi2)
    fmt.Printf("Kartu 2 : (%d,%d)\n", kartu2.Sisi1,
kartu2.Sisi2)

    if bisaDisambung(kartu1, kartu2) {
        fmt.Println("Kedua kartu dapat disambungkan")
    } else {
        fmt.Println("Kedua kartu tidak dapat disambungkan")
    }
}
```

Output:



The screenshot shows a Go IDE with the source code from the previous block. The terminal output is as follows:

```
PS D:\apro\ALPRO2> go run "d:\apro\ALPRO2\laprakmodul18\vo3.go"
Kartu 1 : (2,5)
Kartu 2 : (5,6)
Kedua kartu dapat disambungkan
PS D:\apro\ALPRO2>
```

Overlaid on the terminal is a small window titled "NAMA ITRHOH AN" containing the text:

```
NAMA: ITRHOH ANGGUN PAMUNGKAS
NIM: 109082500117
```


Deskripsi Program:

Program mensimulasikan permainan Gapleh dengan memeriksa apakah dua kartu domino dapat disambungkan. Dua kartu dapat disambungkan jika memiliki minimal satu suit yang sama.

Statement Skema Pemrosesan Sekuensial

4.

Source Code:

```
package main

import "fmt"

var data string
var pos int

func start() {
    pos = 0
}

func maju() {
    pos++
}

func eop() bool {
    return data[pos] == '.'
}

func cc() byte {
    return data[pos]
}

func main() {
    fmt.Print("Masukkan karakter (akhiri dengan .): ")
    fmt.Scan(&data)

    start()

    jumlahKarakter := 0
    jumlahA := 0
    jumlahLE := 0

    var prev byte

    for !eop() {

        jumlahKarakter++

        if cc() == 'A' {
            jumlahA++
        }

        if prev == 'L' && cc() == 'E' {
```

```

        jumlahLE++
    }

    prev = cc()
    maju()
}

frekuensiA := 0.0
if jumlahKarakter > 0 {
    frekuensiA = float64(jumlahA) /
float64(jumlahKarakter)
}

fmt.Println("Jumlah karakter :", jumlahKarakter)
fmt.Println("Jumlah huruf A :", jumlahA)
fmt.Printf("Frekuensi A : %.2f\n", frekuensiA)
fmt.Println("Jumlah kata LE :", jumlahLE)
}

```

Output:

The screenshot shows a VS Code editor with a Go file named `laprakmodul18.go`. The code defines a `main` package with a `start` function that reads input from the user. It then calculates the frequency of the character 'A' and the number of words 'LE' in the input string. The output is displayed in the terminal window.

```

PS D:\lapro\VALPRO2> go run "d:\lapro\VALPRO2\laprakmodul18\val.go"
Masukkan karakter (akhiri dengan .): ALEWALEA.
Jumlah karakter : 9
Jumlah huruf A : 4
Frekuensi A : 0.44
Jumlah kata LE : 2
PS D:\lapro\VALPRO2>

```

Deskripsi Program:

Program mengimplementasikan mesin abstrak karakter yang membaca karakter satu per satu hingga menemukan tanda titik (.) sebagai penanda akhir data. Program kemudian menghitung jumlah karakter yang terbaca, jumlah huruf A, frekuensi kemunculan huruf A terhadap seluruh karakter, serta jumlah pasangan huruf "LE" yang muncul secara berurutan dalam rangkaian karakter.

DAFTAR PUSTAKA

Modul Praktikum Algoritma dan Pemrograman 2. (2025). *Modul 18: Mesin Abstrak*. Telkom University.